

FileSorter

Un tri intelligent, local et configurable
pour les ressources pédagogiques

OCR, extraction multi-format et classification locale avec Ollama / Llama 3.1

Présenté par Samuel Boutin

19 août 2026

FS

Le problème : l'accumulation masque l'information utile

FileSorter transforme un dossier de dépôt en arborescence pédagogique exploitable.

POINT DE DÉPART

Des cours hétérogènes arrivent dans un même dossier : PDF, documents bureautiques, tableaux et scans. Les noms de fichiers ne suffisent pas toujours à comprendre la matière ou l'unité.

Avant

Dépôt/
cours-final.pdf
scan_004.png
tableau.xlsx
devoir.docx

OBJECTIF DU PROJET

Lire le contenu réel, proposer une catégorie, ranger le fichier et mémoriser les validations - sans envoyer les documents vers un service d'IA distant.

Après

Tri/
Informatique/U5/
CEJM/2e année/
Anglais/
Projet/Projets BTS/

Le classement devient une chaîne de traitement reproductible, pas une tâche manuelle ponctuelle.

Une chaîne en 6 étapes, du dépôt au rangement

Chaque étape réduit l'incertitude avant le déplacement définitif du fichier.



Deux modes d'exécution

AUTO

Traitement sans intervention, déclenché par le watcher.

INTERACTIF

Confirmation ou correction avant déplacement du fichier.

La logique complète est centralisée dans `organize()`, appelée directement ou par `watcher.py`.

Chaque format suit le lecteur le plus adapté

Le système combine extraction native et OCR de secours pour couvrir les supports de cours courants.

PDF

.pdf

pdfplumber

OCR si aucun texte détecté

Bureautique

.docx / .xlsx / .xls

python-docx / openpyxl

Lecture paragraphes et cellules

Ouvert & texte

.odt / .txt / .md / .csv

odfpy / lecture UTF-8

Extraction directe du contenu

Images

.png / .jpg / .jpeg / .bmp

Tesseract OCR

Reconnaissance du texte scanné

Garde-fous de performance

Les PDF sont limités aux 2 premières pages et le texte transmis au classifieur est plafonné à 1 500 caractères.

La classification privilégie les signaux les plus explicables

L'IA locale intervient après les informations déjà disponibles dans le projet.



Catégories configurées

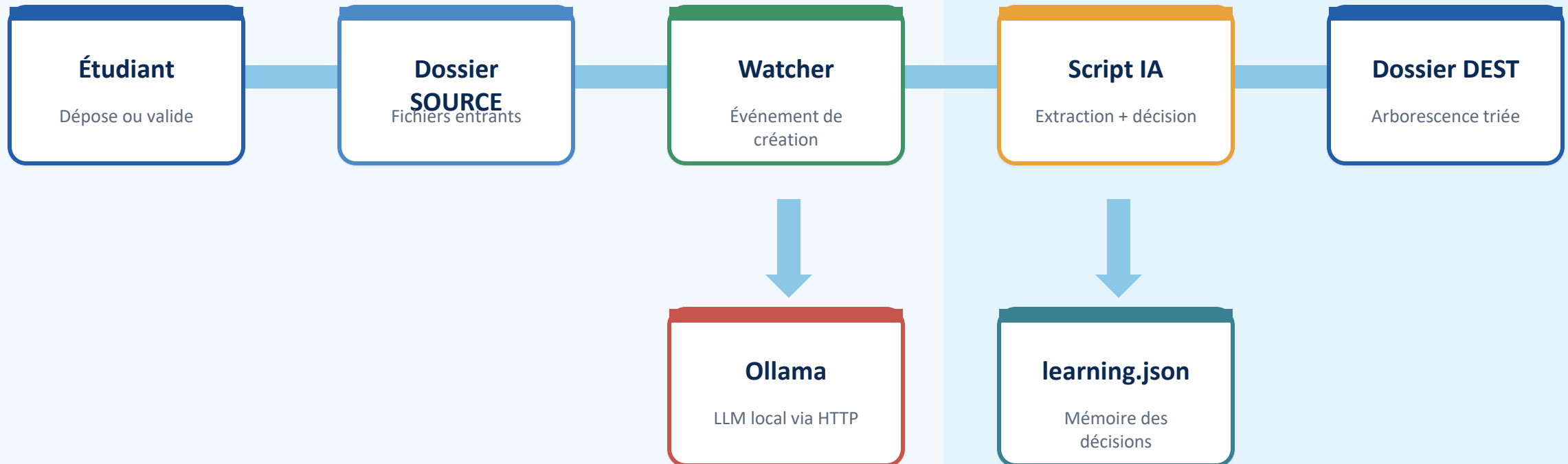
CEJM
Anglais
Maths
Informatique
Culture Générale
Projet
Autre
Non classé

Le choix reste lisible : les catégories et leurs descriptions sont déclarées dans config.py, sans dépendre d'un modèle distant.

À vérifier dans la version actuelle : le retour systématique « Non classé » des règles peut empêcher le repli sur l'IA.

L'architecture reste locale, modulaire et lisible

Le traitement, le modèle et la mémoire d'apprentissage restent sur le poste de l'étudiant.



Aucun document n'a besoin de quitter la machine pour être classé.

Chaque correction enrichit la mémoire du système

L'utilisateur reste maître du classement et enrichit l'historique au fil des usages.

MODE AUTO

Le watcher déclenche `organize()` et déplace le fichier selon la décision calculée.

Usage adapté

Traitement continu lorsque les règles et catégories sont stabilisées.

Principe

Une correction devient une règle réutilisable, sans réentraîner le LLM.

MODE INTERACTIF

1

Proposition

Le système affiche catégorie et sous-catégorie détectées.

2

Confirmation ou correction

L'utilisateur accepte ou saisit une nouvelle catégorie.

3

Mémorisation

La décision, les mots-clés et le statut corrigé sont ajoutés à `learning.json`.

Un socle Python modulaire et configurable

Les responsabilités sont séparées entre orchestration, lecture, surveillance et mémoire.

STRUCTURE DU PROJET

```
FileSorter/  
  README.md  
  requirements.txt  
  src/  
    config.py  
    script.py  
    watcher.py  
    utils/  
      file_reader.py  
      learning.py  
  data/  
    learning_sample.json
```

PARAMÈTRES CENTRAUX

SOURCE / DEST	Chemins d'entrée et de sortie
MODE	auto ou interactive
OLLAMA_URL	API locale sur localhost:11434
MAX_CHARS	1 500 caractères analysés
PDF_PAGE_LIMIT	2 pages extraites
SUBCATEGORIES	U3, U5, U7, années et types

Dépendances principales : requests, watchdog, pdfplumber, python-docx, openpyxl, odfpy, pytesseract, Pillow, langdetect.

La robustesse passe par quatre corrections ciblées

Ces améliorations rendent le prototype plus fiable, portable et démontrable en contexte scolaire.

01

Garantir le repli IA

Retourner None lorsque les règles ne concluent pas, afin que ask_ai() soit réellement appelé.

02

Supprimer l'import circulaire

Déplacer extract_keywords() dans un module utilitaire partagé entre script.py et learning.py.

03

Rendre les erreurs visibles

Remplacer les except silencieux, ajouter journaux, timeouts HTTP et messages d'échec exploitables.

04

Sécuriser le watcher

Attendre la fin de copie, éviter les doubles traitements et empêcher l'exécution simultanée des deux modes.

Ce recul critique ne remet pas en cause le concept : il définit la feuille de route vers une version plus industrialisée.

Le projet mobilise des compétences directement valorisables

Chaque choix technique correspond à une capacité attendue en développement et en exploitation.

Automatisation

Surveillance événementielle, déplacement de fichiers et orchestration d'un flux continu.

IA locale

Construction d'un prompt contraint, appel d'une API locale et combinaison règles + LLM.

Diagnostic

Identification des cas limites, dépendances locales et risques d'exécution concurrente.

Traitement documentaire

Lecture multi-format, OCR, normalisation du texte et limitation des volumes analysés.

Conception logicielle

Configuration centralisée, séparation des responsabilités et persistance JSON.

Documentation

README, mode opératoire, architecture C4 et consignes de lancement.

Une base solide pour industrialiser FileSorter

La valeur du projet tient autant à son utilité immédiate qu'à son potentiel d'évolution.

AUJOURD'HUI

- Une chaîne de tri locale et configurable
- 12 extensions de fichiers prises en charge
- Un mode automatique et un mode supervisé
- Une mémoire simple des validations et corrections
- Une documentation d'installation et d'usage

PROCHAINES ÉTAPES

- Tests unitaires et jeux de documents de référence
- File d'attente robuste pour les nouveaux fichiers
- Base SQLite et traçabilité des décisions
- Embeddings pour rapprocher les contenus similaires
- Interface de supervision et packaging installable

FileSorter automatise une tâche réelle tout en restant compréhensible, local et améliorable.

<http://github.com/sami3737/FileSorter>